Applied Soft Computing 145 (2023) 110569

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

A method for finding numerical solutions to Diophantine equations using Spiral Optimization Algorithm with Clustering (SOAC)

Novriana Sumarti [a,*], Kuntjoro Adji Sidarto [a], Adhe Kania [a], Tiara Shofi Edriani [b], Yudi Aditya [c]

[a] Industrial and Financial Mathematics Research Group, Institut Teknologi Bandung, Ganesha 10 Bandung 40132, Indonesia
[b] Mathematics Master Study Program, Institut Teknologi Bandung, Ganesha 10 Bandung 40132, Indonesia
[c] Computational Science Master Study Program, Institut Teknologi Bandung, Ganesha 10 Bandung 40132, Indonesia

ARTICLE INFO

Article history:
Received 15 May 2021
Received in revised form 20 May 2023
Accepted 17 June 2023
Available online 26 June 2023

MSC:
11Y50
65K10
90C59

Keywords:
Polynomial and exponential Diophantine equations
The Markoff–Hurwitz equation
Root finding algorithm
Optimization

ABSTRACT

Diophantine equations are equations containing two or more unknowns, such that only the integer solutions are required. To find solutions of these equations numerically, we can be performed by solving an optimization problem using a metaheuristic method. In this paper, the Spiral Optimization Algorithm with Clustering (SOAC) method is proposed to find solutions to Diophantine equations in the form of polynomial, exponential, and also linear and nonlinear systems of equations. In the implementation of the method on solving some existing benchmark problems, the goal of simulation is to find all solutions only in a single run and in a short period of time. Appropriate values of required parameters are selected during the simulation. Results shows satisfactory in solving four problems in polynomial equations, four problems in exponential equations, and three problems in systems of linear and nonlinear equations. In most of cases, the results yield the same with the analytical or numerical solutions in the reference papers, and in some cases the results give more solutions.

1. Introduction

The general definition of a Diophantine equation is an equation of two or more unknowns whose the desired solutions are integer numbers. There is a well-known challenge in Hilbert's 10th problem whether there exist algorithms or formulas for determining the solutions of arbitrary Diophantine equations. Some researches of Number Theory investigate the size of the set of solutions and search for upper bounds for the size of finite solutions set, if it is available, or figure out the structure of the set of the non-finite solutions set. The difficulty increases when the dimension of the solutions is larger or there are more than two equations in a connected system.

Some of current research that had been conducted were discussing random Diophantine equations [1], the class number of a quadratic Diophantine equation [2], Diophantine triples [3] and linear Diophantine equations in several variables [4]. Authors in [1] considered additive Diophantine equations of degree k in s variables and established that whenever $s \geq 3k+2$ then almost all such equations satisfy the Hasse principle. This principle states that certain types of equations have a rational solution if and only if they have a solution in the real numbers and in the p-adic numbers for each prime p. The Diophantine equation's coefficients were chosen at random.

Author in [2] defined the class number of a quadratic Diophantine equation so that it can be viewed as a measure of the obstruction of the local–global principal for quadratic Diophantine equations. The method for computing the class number was using the mass formula of Minkowski for lattice translation.

The form of Diophantine equations that have integer numbers as the solution can be found everywhere. This is an important type of equation that can be found in many fields. Papers [5] implemented the Diophantine equations in defining the security of the public-key cipher system, and [6] tried to identify its source of in-secures. One of the conclusion from [7] claimed that the formal decision problem framework for a bounded rational information processing system can be constructed in systems of linear Diophantine inequalities. So it can be used to propose a change in the market equilibrium conditions.

A particular Diophantine equation can be developed from the study of Regular Languages, which is a concept in computability courses that is used in parsing and designing programming languages. Authors in [8] proposed method for testing whether a regular language is semi-geometrical or not and whether it is

* Corresponding author.
E-mail address: novriana@itb.ac.id (N. Sumarti).

https://doi.org/10.1016/j.asoc.2023.110569
1568-4946/

geometrical or not, in the case where the minimal automaton or machine admits a strongly connected diagram. It dealt with a system of linear Diophantine equations, whose form of $Ax = b$, where $A = (a_{ij})$ is an $m \times n$ matrix with integer entries, b is an $m \times 1$ column vector with integer components and x is an $n \times 1$ solution vector with integer components. Here the existence of a solution is concerned.

Authors in [9] presented an illustrative example proving the Diophantine nature of cyclic scheduling models, where the observed activities are repeated an indefinite number of times. These problems can be found in many domains; such that manufacturing, digital signal processing, workforce scheduling, train timetabling, aircraft routing and scheduling, and reservations. Solving a system of repetitive manufacturing processes, consisting of a set of processes sharing common resources P_1, P_2 and P_3, can be written as a Diophantine problem for finding the frequencies y_{P_1}, x_{P_2} and z_{P_3} of processes that satisfy

$$3y_{P_1} + 3x_{P_2} = T + z_{P_3}.$$

where T is the periodicity of the system of concurrently executed cyclic processes and its value is given.

In the Number Theory, there are also Diophantine inequalities and the Diophantine approximation, where the latter is a study to approximate real numbers by rational numbers. Authors in paper [10] applied the theory of metric Diophantine approximation in order to develop new approaches in the Interference Alignment, a technical concept in wireless communication networks that attempts to align interfering signals in time, frequency, or space.

In [11], the increase in aggregated peak allocated capacity was estimated using E-Diophantine solutions consisting two heuristics of polynomial complexity: E-Diophantine-W (Weighted) and E-Diophantine-UW (Unweighted). It reduced the E-Diophantine computational load from exponential to polynomial (cubic) at a low estimation error probability cost.

Currently, there have been research conducted to solve the equations numerically. In the first approach, a method could attempt to cover almost everything in a systematic way by exploring as many candidate solutions as possible. In another approach, the method could use randomness in the process, so the risk of uncovering solutions will exist if the number of repetitions in executing the algorithm is not enough.

Using the second approach, authors in [12] had proposed some approaches in solving Polynomial Diophantine equations in the forms of

$$x_1^k + x_2^k = N, \tag{1}$$

$$x_1^2 + x_2^2 + \cdots + x_n^2 = M, \tag{2}$$

where N, M are particular integers, k from 2 to 15, and n are from 2 to 12. They applied some well-known optimization methods, such as the Genetic Algorithm (GA), the Simulated Annealing (SA), the Particle Swarm Optimization (PSO), the Steepest Ascent Hill Climbing, the Connectionist Network approach and the Ant Colony Optimization. Paper [13] used the discrete version of PSO to numerically solve a system of linear Diophantine equations.

This paper proposes a numerical method with an optimization approach containing a well-defined objective function, so that the obtained solutions approximate the roots of the observed equations. We use a modified Spiral Optimization Algorithm (SOA) to find all possible solutions of some types of Diophantine equations. Developed by Tamura and Yasuda in 2010 [14], Spiral Optimization Algorithm (SOA) is a metaheuristic method for finding numerical solutions of optimization problems. The inventors were inspired by geometrical spiral appearances commonly occurred in a conch shell, a water movement at a whirlpool, a galaxy-arms containing starts, gas and dusts, and others. Based on the two-dimensional spiral rotation, the algorithm had been modified by generalizing the model to an n-dimensional spiral model [15]. The SOA model generates spiral forms containing centers, which are dynamically updated in each iteration.

The SOA method have been implemented for solving optimization problems and root finding problems. In paper [16], this method was applied to solve a multi-objective optimization problem of the Combined Economic and Emission Dispatch (CEED) problem, so the schedule of generators can result in both minimum fuel costs and emission levels, simultaneously, while satisfying the load demand and operational constraints. In paper [17], the combination of the SOA method with the k-means and oscillation methods was used to solve the clustering problem for data mining, so it can increase the diversity of search to further improve the clustering result. Paper [18] had applied the method in finding approximated values of many parameters needed in the dynamical system of Deposit and Loan volumes of a bank that complying a historic data. This dynamical system was used as a potential analytical tool for approximating the stability condition of banking.

Some modifications from the original SOA method had been done for enriching its capability and enhancing its efficacy. Instead of obtaining real solutions only, the modified SOA in paper [19] could obtain mixed forms (real and integer) solutions. The paper also proposed the usage of the Sobol sequence in generating candidates of solutions, instead of commonly used normal random generation, so it could provide solutions being evenly distributed inside the domain. In [20], the original method was implanted by a new procedure for clustering, so it is called SOAC (Spiral Optimization Algorithm with Clustering). The SOAC method is potentially as a reliable and efficient method due to the success result in finding all solutions of some stiff problems only for one execution of the program. Papers [21,22] had also shown satisfactory performance of the SOAC method in solving multimodal optimization problems and finding complex roots of nonlinear equations systems, respectively.

In this paper, we use SOAC method to find all possible solutions of some types of Diophantine equations. In the implementation of other methods in previously mentioned papers, most of them need a large number of repetitions in executing the methods, in order to find feasible solutions one by one. Our method is not based on randomness, so if the parameters are not changed, the result will be the same when the program is rerun. Using the appropriately chosen of parameters' values, we can find available solutions using a single run computation. More research papers containing particular Diophantine equations will be observed later in Section 3 and their results will be compared to our results.

In the next section, the formulation of the problem in the optimization approach is explained. Using this formulation, the problem is ready to be solved by the algorithm steps of SOAC method. The implementation of the method on selected Diophantine equations is presented in Section 3. In the beginning, the method solves some problems in simple forms, such as polynomial and exponential equations. Furthermore, the method is used to solve problems in systems of linear and nonlinear equations, which is difficult to be solved analytically. Finally the discussion concerning the results and conclusions are written in Sections 4 and 5.

2. Optimization method for solving Diophantine equations

We utilize the optimization approach for finding the roots of the observed Diophantine Equations. Later it needs a objective

function derived from the error function to be minimized. However, the definition of this objective function is not trivial. We elucidate this approach later.

Consider a system of linear or nonlinear equations of the form as follows.

$$f_i(\mathbf{x}) = 0, i = 1, 2, \ldots, m, \tag{3}$$

where $\mathbf{x}$ is a vector with size n, f_i continuous function of domain $D \subset R^n$.

$$D = \{(x_1, x_2, \ldots, x_n) : a_i \leq x_i \leq b_i, i = 1, 2, \ldots, n\},$$

where a_i and b_i are respectively the lower and upper bounds of x_i. So there are m equations and n variables. The solution $\mathbf{x}^* \in D$ satisfying the system are the roots of system (3) from a defined searching domain D. The roots finding problem of system (3) can be constructed into a single-objective optimization problem so the obtained roots are equivalent with the optimal solutions of optimization problem.

Motivated by the fitness function in [23], we define the objective function for the optimization problem as follows.

$$F(\mathbf{x}) = \frac{1}{1 + \sum_{i=1}^{m} |f_i(\mathbf{x})|}. \tag{4}$$

It is straightforward that $F(\mathbf{x})$ has maximum value 1. If there is $\mathbf{x}^*$ such that $F(\mathbf{x}^*) = 1$, then $\mathbf{x}^*$ is the global maximum of $F(\mathbf{x})$. Consequently, $f_i(\mathbf{x}^*) = 0$ for $i = 1, 2, \ldots, m$, which means $\mathbf{x}^*$ is the root of the system of equations. Therefore finding $\mathbf{x}^*$ satisfying $F(\mathbf{x}^*) = 1$ is equivalent with finding the roots of the system of linear or nonlinear equations (3). Based on our experience, this objective function is better than other commonly used objective functions in the maximization process, because its special form can distinguish among solutions $\mathbf{x}$ that are very close, for example, those are numbers with the same values up to 3 decimal places.

Many nonlinear optimization problems involve many variables under few constrains, so this highly nonlinearity problems require multimodal objective functions. In this case, only global search algorithms, such as metaheuristic methods, are suitable for obtaining optimal solutions commonly needed in the conventional approximation methods. A demanding requirement of the differentiability of objective functions is not needed in these methods so the problem being solved could be continuous or discrete.

In the SOA model, it generates spiral forms containing centers, which are dynamically updated in each iteration. The solution $\mathbf{x}^*$ to the problem is obtained from one of the dynamical centers that satisfy the defined objective function, such as (4) in our research. In [24], it has been proven that the center $\mathbf{x}^*(k), k = 1, 2, \ldots, k_{max}$ is unique and globally asymptotically stable, where k_{max} is the maximum number of iteration. The dynamical centers generated along the iteration will converge to the solution $\mathbf{x}^*$ with rate $r(k), 0 < r(k) < 1$. The authors suggested a parameter setting of $r(k)$ that can appropriately balance the diversification process to the intensification process in finding the candidates of centers. In [25], the conditions and settings of the composite rotation matrix $R(\theta), 0 < \theta < 2\pi$, the rate $r(k)$ and initial points $x_i(0), i = 1, \ldots, m$ were mathematically analyzed to ensure the convergence of the SOA algorithm to the obtained solution. The definitions of these variables are explained in the next subsection. We also explain the SOAC method, which is the SOA method that is modified by schemes on clustering and obtaining the mixed forms (real and integer) solutions.

2.1. Method of Spiral Optimization Algorithm with Clustering (SOAC)

The SOAC method contains three main phases; clustering, optimization and selection. In the clustering phase, a number of initial points is generated using Sobol Sequence and then these points are grouped into a number of clusters, where each cluster has a center and its radius of coverage region. Each cluster's center has the best estimated values among existing points on the boundary of the cluster. The next phase is the Optimization where the spiral technique on each cluster is implemented to have candidates which have better values than the original cluster's center. Finally, the Selection phase finds the best values among candidate points of all clusters so they are concluded as the solutions of the problem.

2.2. Algorithm steps of SOAC

Now we consider a system of linear or nonlinear equations $\mathbf{f}(\mathbf{x}) = 0$(3) and the objective function $F(\mathbf{x})$ of the optimization problem (4). The following algorithm is our SOAC method firstly proposed in [20], where the steps of algorithm of SOAC is modified for solving integer roots of Diophantine Equations.

Steps of the algorithm

Input

$m_{cluster}$: the number of intended clusters, $m_{cluster} \geq 2$,
γ: the cutting off value for zero value of f, $0 < \gamma < 1$,
$k_{cluster}$: the maximum iteration for updating the center of clusters,
r: the convergence rate of spiral algorithm, $0 < r < 1$,
θ: the rotation angle of spiral algorithm, $0 < \theta < 2\pi$,
m: the initial number of points in the search area,
k_{max}: the maximum iteration for spiral algorithm,
ϵ: the bound of tolerance for accepting a root, $0 < \epsilon < 1$,
δ: the bound for distinguishing between two roots, $0 < \delta < 1$.

Clustering Phase

1. Generate initial points, using Sobol sequence, $\mathbf{x}_i(0) \in R^n$, $i = 1, 2, \ldots, m_{cluster}$ in the search region $\mathbf{D}$, where $\mathbf{D} = [a_1, b_1] \times [a_2, b_2] \times \cdots \times [a_n, b_n] \subset R^n$.
2. Set $k = 0$.
3. For $i = 1, 2, \ldots, m_{cluster}$, define rounded points $\mathbf{q}_i(0) = \left(q_{i,1}(0), q_{i,2}(0), \ldots, q_{i,n}(0)\right)^T$ such that $q_{ij}(0) = \text{round}\left(x_{ij}(0)\right)$.
4. Set $\mathbf{x}^* = \mathbf{q}_{i_g}(0)$ where $i_g = \text{argmax}_i F(\mathbf{x}_i(0)), i = 1, 2, 3, \ldots, m_{cluster}$.
5. Store $\mathbf{x}^*$ as the center of first cluster C_1 with radius $\rho_1 = \frac{1}{2} \min |b_l - a_l|, l = 1, 2, \ldots, m_{cluster}$.
6. For $i = 1, 2, 3, \ldots, m_{cluster}$ do
   If $F(\mathbf{x}_i) > \gamma$ and $\mathbf{x}_i$ is not an existing cluster's center, then examine whether $\mathbf{x}_i$ can be a new cluster's center or not by putting it as an input $\mathbf{y} = \mathbf{x}_i$ into the clustering function below $C_F(\mathbf{y})$. The output of this step is a new points $\mathbf{x}_i(k)$ for an increment k.

   Clustering Function $C_F(\mathbf{y})$.

   a. Find a cluster with its center being the closest to $\mathbf{y}$.

   b. Let C be that cluster with center $\mathbf{x}_C$.

   c. Set $\mathbf{x}_t$ as the mid-point between $\mathbf{y}$ and $\mathbf{x}_C$,

   d. Evaluate values of $F(\mathbf{y})$, $F(\mathbf{x}_C)$ and $F(\mathbf{x}_t)$

   - If $F(\mathbf{x}_t) < F(\mathbf{y})$ and $F(\mathbf{x}_t) < F(\mathbf{x}_C)$,
     Set $\mathbf{y}$ as a center of new cluster and its radius is the distance between $\mathbf{y}$ and $\mathbf{x}_t$.

- Else, if $F(\mathbf{x}_t) > F(\mathbf{y})$ and $F(\mathbf{x}_t) > F(\mathbf{x}_C)$,
  Set $\mathbf{y}$ as a center of new cluster and its radius is the distance between $\mathbf{y}$ and $\mathbf{x}_t$. Examine whether $\mathbf{x}_t$ can be another new cluster's center by calling again the Clustering Function with input $\mathbf{y} = \mathbf{x}_t$.
- Else, if $F(\mathbf{y}) > F(\mathbf{x}_C)$, set $\mathbf{y}$ as the center of cluster C, because $\mathbf{y}$ has dominated $\mathbf{x}_C$.

e. Change the radius of C to the distance between $\mathbf{y}$ and $\mathbf{x}_t$.

7. For $i = 1, 2, 3, \ldots, m_{cluster}$ and $\mathbf{x}_i$ are in the search domain, define again points $\mathbf{q}_i(k)$ such that $q_{ij}(k) = \text{round}\left(x_{ij}(k)\right)$, $j = 1, 2, \ldots, n$. Make sure that $\mathbf{q}_i(k)$ are also in the search domain.
8. Set $\mathbf{x}^* = \mathbf{x}_{i_g}$ where $i_g = \underset{i}{\text{argmax}} F(\mathbf{q}_i(k))$, .
9. Update $\mathbf{x}_i$ by doing the spiral transformation

$$\mathbf{x}_i(k+1) = \mathbf{S}_n(r,\theta)\mathbf{x}_i(k) - (\mathbf{S}_n(r,\theta) - \mathbf{I}_n)\mathbf{x}_p,\ i = 1, 2, 3, \ldots, m_{cluster},$$

where $\mathbf{I}_n$ is an $n \times n$ identity matrix, and $S_n(r,\theta) = rR^{(n)}(\theta_{12}, \theta_{13}, \ldots, \theta_{n-1,n})$. Here $R^{(n)}(\theta_{12}, \theta_{13}, \ldots, \theta_{n-1,n})$ is the multiplication among all rotation matrices $R^{(n)}_{i,j}(\theta_{i,j})$ for all $i \neq j$, $1 \leq i, j \leq n$.

10. Repeat Steps 6 to 9 until $k = k_{cluster}$.
11. Set $\mathbf{x}_{C_i}$ for $i = 1, 2, \ldots, n_c$ as the center points of obtained clusters, where $x_{C_{i,j}} = \text{round}\left(x_{ij}\right)$.

Spiral Optimization Phase in each Cluster

12. From steps 1–11, we have clusters C_i with center $\mathbf{x}_{C_i}$ and radius ρ_i. Perform SOA in each cluster $i = 1, 2, \ldots, n_c$ with parameters m, r, θ, k_{max}, so the solution candidates are obtained for each cluster. The search domain of each cluster is $\mathbf{D}_i = \left[x_{C_{i,1}} - \rho_i, x_{C_{i,1}} + \rho_i\right] \times \left[x_{C_{i,2}} - \rho_i, x_{C_{i,2}} + \rho_i\right] \times \cdots \times \left[x_{C_{i,n}} - \rho_i, x_{C_{i,n}} + \rho_i\right]$.

Selection of Roots

13. Choose root candidate $\mathbf{x}$ that satisfy condition $1 - F(\mathbf{x}) \leq \epsilon$ only.
14. Suppose from Step 13 there are n_g candidate roots. We want to eliminate identical roots.
    For $i = 1, 2, \ldots, n_g - 1, j = i + 1, i + 2, \ldots, n_g$,
    If distance $\|\mathbf{x}_i - \mathbf{x}_j\| > \delta$, then choose both $\mathbf{x}_i$ dan $\mathbf{x}_j$ as roots.
    If $\|\mathbf{x}_i - \mathbf{x}_j\| \leq \delta$, and if $F(\mathbf{x}_i) \geq F(\mathbf{x}_j)$, then choose $\mathbf{x}_i$ as a root. Else, choose $\mathbf{x}_j$ as a root.

Output

$\mathbf{x}$ are the roots of equation $\mathbf{f}(\mathbf{x}) = 0$.

Remember that the solutions of the problems being solved are intended to be integer. We know the obtained values of the numerical methods are mostly in real number. In our method, the values of particular variables that are the candidates of solutions will be rounded into the nearest integer. However, their values are kept in real number when the spiral iteration is performed. In the above algorithm in steps 3 and 7, variable $\mathbf{q}$, an integer vector obtained from rounding $\mathbf{x}$, is used for finding the maximum value of the objective function $F(\mathbf{q})$, but the real vector $\mathbf{x}$ is still used in the spiral rotation at the next steps. In the Selection phase, the obtained solutions $\mathbf{x}$ are the best integer values satisfying the Diophantine equations without containing any errors.

3. Implementation of the method

In the implementation process to solve the Diophantine equations, we work using MATLAB R2016b in a CPU Intel Core i5-83000H 2.3 GHz, 8 Gb RAM, and using MATLAB R2016a in a CPU Intel Core i5-8250U @1.6 GHz 3.4 GHz, 4 GB RAM. First we begin with the simplest form of polynomial equation, then finally the system of linear and nonlinear equations being solved. For some problems, the solutions written in this paper are only some part of the obtained result. Sometime by adjusting the parameters to higher values, we can find more solutions of certain problems. However we stop trying to adjust when the running time of simulation is more than 300 s.

Table 1
Parameters for Problems 1–4.

Problems	1	2a	2b	3a	3b	4
Clustering and selection						
$m_{cluster}$	350	1000	2000	200	100	2000–8500
$k_{cluster}$	10	20	20	20	20	10-40
γ	0.01	0.1	0.1	0.1	0.1	0.001–0.0001
ϵ	10^{-7}	10^{-5}	10^{-5}	10^{-5}	10^{-5}	10^{-7}
δ	0.1	0.01	0.01	0.01	0.01	0.1
Optimization						
m	30	50	50	50	50	60-400
k_{max}	10	50	50	50	50	10-50
r	0.95	0.95	0.95	0.95	0.95	0.95
θ	$\pi/4$	$\pi/4$	$\pi/4$	$\pi/4$	$\pi/4$	$\pi/4$

3.1. Polynomial Diophantine equations

Problem 1. The simplest form of the equation with two variables is

$$ax + by = c, \text{ where } a, b, c \in Z. \tag{5}$$

Based on [26], if a, b, c are positive integer, then the solution has form of

$$x = x_0 + \left(\frac{b}{d}\right)n, \text{ and } y = y_0 + \left(\frac{a}{d}\right)n, n \in Z, \tag{6}$$

where $d = gcd(a, b)$, gcd is the greatest common divisor. For the case $a = 15, b = 11$ and $c = 12$, for $-50 \leq x \leq 50$ and $-50 \leq y \leq 50$, only seven solutions are obtained and the same with the solutions from some theoretical papers including [26]. The SOAC parameters used for finding these solutions are written in Table 1. The given domain for (x, y), and the obtained solutions calculated for 0.3488 s are shown in Table 2.

Problem 2. Two quadratic multivariable equations in the form of (1) are picked to be solved. The equations are

(a) $x_1^2 + x_2^2 + x_3^2 + \cdots + x_9^2 = 720$,

(b) $x_1^2 + x_2^2 + x_3^2 + \cdots + x_{10}^2 = 956$,

are solved using the parameters' values in Table 1. Written in paper [27], Problem 2(a) has solution (1, 1, 1, 2, 2, 14, 12, 12, 15) and Problem 2(b) has solution (1, 1, 1, 1, 1, 1, 1, 15, 20, 18). We expect that there are more obtained solutions that are not written in that paper. Using variables' domain in Table 2, we obtain 84 solutions in 2.5102 s for the first problem and 96 solutions in 3.2316 s for the second problems. Some of them are the same values but in different orders. Table 2 shows respectively only six of the total. Note that this result is far from exhaustive yet, moreover if we refine the clusters more by altering the values of parameters.

Problem 3. Two high order polynomials in the form of (2), are picked to be solved, which are

(a) $x_1^3 + x_2^3 = 1008$,

(b) $x_1^9 + x_2^9 = 1000019683$.

Table 2
Solutions for Problems 1–2.

Problems	Given domain and obtained solutions (x_i, y_i)
1	$D = \{(x, y) : -50 \le x \le 50, -50 \le y \le 50\}$ (25,−33),(3,−3),(−19,27),(−30,42),(−8,12),(14,−18),(36,−48).
2a	$D = \{(x_1, x_2, \ldots, x_9) : 1 \le x_i \le 26, i = 1, 2, \ldots, 9\}$ (1,2,3,5,5,6,10,14,18),(2,3,3,3,4,10,11,14,16),(3,4,4,6,7,9,12,12,15), (4,4,6,7,7,7,9,10,18),(5,6,6,7,7,8,11,12,14), (5,7,7,8,9,9,9,11,13).
2b	$D = \{(x_1, x_2, \ldots, x_10) : 1 \le x_i \le 26, i = 1, 2, \ldots, 9\}$ (1, 2, 4, 4, 5, 9, 14, 14, 14, 15), (2, 2, 3, 3, 6, 6, 7, 14, 17, 18), (3, 3, 3, 4, 5, 6, 13, 13, 15, 17), (4, 5, 6, 6, 6, 7, 9, 9, 14, 20), (5, 6, 8, 8, 9, 9, 11, 12, 12, 14), (5, 7, 7, 9, 9, 9, 10, 11, 12, 15).

Table 3
Parts of solutions to Problem 4.

n	k	Solution	Time (s)
3	1	(3, 3, 3), (6, 3, 3), (15, 6, 3), (39, 15, 3), (87, 15, 6), (102, 39, 3), (267, 102, 3).	21.05
3	3	(1, 1, 1), (2, 1, 1), (5, 2, 1), (13, 5, 1), (29, 5, 2), (34, 13, 1), (89, 34, 1), (169, 29, 2), (194, 13, 5), (233, 89, 1), (433, 5, 29)	229.01
4	1	(2, 2, 2, 2), (6, 2, 2, 2), (22, 6, 2, 2), (82, 22, 2, 2), (262, 22, 6, 2).	8.65
4	4	(1, 1, 1, 1), (3, 1, 1, 1), (11, 3, 1, 1), (41, 11, 1, 1), (131, 11, 3, 1).	2.85
5	4	(2, 1, 1, 1, 1), (7, 2, 1, 1, 1), (26, 7, 1, 1, 1), (55, 7, 2, 1, 1), (97, 26, 1, 1, 1).	9.79
6	3	(2, 2, 1, 1, 1, 1), (4, 2, 1, 1, 1, 1), (10, 4, 1, 1, 1, 1), (11, 2, 2, 1, 1, 1), (23, 4, 2, 1, 1, 1).	3.33
7	2	(2, 2, 2, 1, 1, 1, 1), (6, 2, 2, 1, 1, 1, 1), (15, 2, 2, 2, 1, 1, 1), (22, 6, 2, 1, 1, 1, 1), (47, 6, 2, 2, 1, 1, 1).	7.38
8	1	(4, 2, 2, 2, 1, 1, 1, 1), (14, 4, 2, 2, 1, 1, 1, 1), (31, 4, 2, 2, 2, 1, 1, 1).	7.11
9	6	(2, 1, 1, 1, 1, 1, 1, 1, 1), (4, 1, 1, 1, 1, 1, 1, 1, 1), (11, 2, 1, 1, 1, 1, 1, 1, 1), (23, 4, 1, 1, 1, 1, 1, 1, 1).	3.32
10	1	(4, 4, 3, 1, 1, 1, 1, 1, 1, 1), (8, 4, 3, 1, 1, 1, 1, 1, 1, 1), (13, 4, 4, 1, 1, 1, 1, 1, 1, 1).	19.87

The parameters' values being used are shown in Table 1. Written in paper [27], Problem 3(a) and 3(b) respectively have solution (2, 10) and (3, 10). Using domain $D = \{(x_1, x_2) : 1 \le x_i \le 10, i = 1, 2\}$, we obtain the same solutions respectively in 0.1239 s and 0.0877 s. Attempts to find other solutions have been done by altering lower and upper bounds of the domain D and the values of parameters, but no other solution is available.

Problem 4. Markoff–Hurwitz Equations [28] are the form of

$$x_1^2 + x_2^2 + \cdots + x_n^2 = kx_1x_2 \ldots x_n, \tag{7}$$

with k is a positive integer and $n \ge 3$. Solution $(x_1, x_2, \ldots, x_n)$ to Eq. (7) with integer $x_i > 0$ for all i, are called Markoff numbers. The equations were first studied by Hurwitz who generalized the Markoff equation proposed in 1879, which is

$$x^2 + y^2 + z^2 = 3xyz. \tag{8}$$

In Table 1, the parameters are chosen for solving (7) for different n and k. Note that the domain of x_is are also customized for each cases, which are set to be wider for first few indices i. For example in case $n = k = 3$, firstly we set $D = \{(x_1, x_2, x_3) : 1 \le x_1 \le 200, 1 \le x_2 \le 40, 1 \le x_3 \le 5\}$ and $m_{cluster} = 2000$, there are solutions obtained for 7.22 s, where some of them are in first seven solutions in Table 3. When we set $D = \{(x_1, x_2, x_3) : 1 \le x_1 \le 500, 1 \le x_2 \le 400, 1 \le x_3 \le 50\}$ and a larger parameter $m_{cluster} = 5000$, there are two additional solutions (233, 89, 1), (433, 29, 5) with the running time is 229.006 s. Then there are totally 31 solutions, where many of them have the same values but different positions. Table 3 shows some part of obtained solutions, where other order combinations of a solution are not written. For some other particular values (n, k), we have the same conclusion as in [28], where no solution exists.

3.2. Exponential Diophantine equations

Problem 5. From [29], the exponential form of the equation is as follows.

$$x^2 + 7^a 11^b = y^n, \text{ where } gdc(x, y) = 1, n \ge 3. \tag{9}$$

Paper [29] gave analytical formulas of the solutions. Below are two considered cases for particular values of a and b.

(a) For $a = 1$ and $b = 0$, which is called Ramanujan–Nagell equation, the equation becomes $x^2 + 7 = y^n$. Having the same solutions as in [29], there are seven positive integer solutions (x, y, n), which is obtained by a single execution running for 8.9269 s. The parameters used in the computation are in Table 4. For given domain of the problem, the obtained solutions are shown in Table 5.

(b) For $a = 0$, the equation becomes $x^2 + 11^b = y^n$. Specially for $n = 3$ and the given domain for positive integers (x, y, b), there are four solutions which are the same as in [29], and they are shown in Table 5. The method using parameters values in Table 4, and only one execution is needed running for 90.1795 s. Note that the value for the cutting off value for zero value of the objective function should be $\gamma = 10^{-77}$, which is extremely near to zero, in order to get solution (9324, 443, 3) particularly.

Problem 6. From [30], another form of exponential equation is following.

$$x^2 + 2^a 11^b = y^n, \text{ where } x \ge 1, y \ge 1, gcd(x, y) = 1, \\ n \ge 3, a \ge 0, b \ge 0. \tag{10}$$

Table 4
Parameters for Problems 5–8.

Problems	5(a)	5(b)	6(a)	6(b)	7	8	9(a)	9(b)
Clustering and selection								
$m_{cluster}$	4000	20 000	600	15 000	30 000	500	400	1500
$k_{cluster}$	40	50	20	20	20	30	20	20
γ	10^{-5}	10^{-77}	0.001	0.01	0.1	0.1	0.0001	0.001
ϵ	0.001	10^{-5}	10^{-5}	10^{-7}	10^{-5}	10^{-7}	10^{-7}	10^{-7}
δ	0.1	0.01	0.01	0.001	0.01	0.1	0.1	0.1
Optimization								
m	150	200	100	100	50	20	100	20
k_{max}	30	50	20	20	50	20	20	10
r	0.95	0.95	0.95	0.95	0.9	0.95	0.95	0.95
θ	$\pi/16$	$\pi/16$	$\pi/4$	$\pi/4$	$\pi/60$	$\pi/4$	$\pi/4$	$\pi/4$

(a) Especially for $n = 3$ and domains $1 \le x \le 20, 1 \le y \le 20$, there are six analytical solutions reported in [30] with a conditional greatest common divisor, $gcd(x, y) = 1$. Using the parameters values in Table 4 and a given domain in Table 5, we obtain the same six solutions (x, y, a, b) and also additional two solutions if the condition *gcd* is not applied. For only one execution, the running time is 0.9455 s. As written in Table 5, the additional solutions are $(2, 2, 2, 0)$ and $(16, 8, 8, 0)$.

(b) For $n = 4$, there is only one analytical solution in [30], which is $(x, y, a, b) = (7, 3, 5, 0)$. The result from the numerical method gives four additional solution if the conditional $gcd(x, y) = 1$ is not applied. The solutions are in Table 5 and the running time is 199.3303 s. We can find other solutions for other attempts with different domains so the solutions are not exhausted yet.

Problem 7. In [31], the following equation

$$2^k + nx^2 = y^n, \text{ where } x \ge 0, y \ge 0, gcd(nx, y) = 1. \quad (11)$$

has only one analytical solution, $(x, y, k) = (21, 11, 3)$ for $n = 3$. There are more solutions obtained from the numerical method using parameters values in Table 4 and without the condition of *gcd*. The solutions are shown in Table 5, which are obtained with running time 21.7346 s .

Problem 8. From [32], one of the exponential equations being solved is the form of

$$5^{x_1} + 5^{x_2} = 3^{x_3} + 7^{x_4}, \text{ where } -1 \le x_i \le 10, i = 1, 2, 3, 4. \quad (12)$$

The paper [32] found nine solutions numerically by using the Particle Swarm Optimization (PSO). The time needed from 10 executions was ranging from 21.99 to 63.686 s. Having used parameters values in Table 4 and the given domain in Table 5, one execution of our method running for 0.9519 s yields nine solutions at once, which are written in Table 3.

Problem 9. The Pell equation is a second order Diophantine equation that has the form of

$$x^2 - ny^2 = k, \text{ where } n \in Z^+, \text{ and given } k \in Z. \quad (13)$$

It is called the positive Pell Equation if $k > 0$, or the negative Pell Equation if $k < 0$. Paper [33] found the analytical solutions of the system of Pell equations (14)–(15) for p is a prime number.

$$x^2 - 24y^2 = 1, \quad (14)$$

$$y^2 - pz^2 = 1. \quad (15)$$

The solutions exist only for $p = 2, 11$. Define Problem 9(a) for $p = 2$, and Problem 9(b) for $p = 11$. Using parameters values in Table 4, the algorithm finds the unique solution for each problem, which are shown in Table 5. In only a single execution for each problem, the computation takes 1.6590 s in Problem 9(a) and 0.5054 s in Problem 9(b).

3.3. Systems of Diophantine linear and nonlinear equations

Problem 10. From [13], the system of linear equations being discussed is the following.

$$\begin{aligned}
f_1(\mathbf{x}) &= x_1 - 1 = 0,\\
f_2(\mathbf{x}) &= 3x_1 + x_2 - 6 = 0,\\
f_3(\mathbf{x}) &= 4x_1 + 3x_2 + x_3 + x_5 - 15 = 0,\\
f_4(\mathbf{x}) &= 3x_1 + 4x_2 + 3x_3 + x_4 + x_5 + x_6 - 20 = 0,\\
f_5(\mathbf{x}) &= 3x_2 + 4x_3 + 3x_4 + x_5 + x_6 + x_7 - 15 = 0,\\
f_6(\mathbf{x}) &= 3x_3 + 4x_4 + x_6 + x_7 - 6 = 0,\\
f_7(\mathbf{x}) &= 3x_4 + x_7 - 1 = 0
\end{aligned}$$

where $\mathbf{x} = (x_1, x_2, \ldots, x_7)^T \in Z^7$ and $D = \{(x_1, x_2, \ldots, x_7) : -10 \le x_i \le 10, i = 1, 2, \ldots, 7\}$. Using the Discrete Particle Swarm Optimization (PSO), paper [13] gave a solution with the running time at least about 100 s. Using parameters values in Table 6, the result gives the same solution shown in Table 7, and it needs only 11.8086 s of running time.

Problem 11. From [32], the nonlinear equations system being considered is as follows.

$$\begin{aligned}
f_1(\mathbf{x}) &= 5x_1 + 10x_2 - 5x_3 + x_5^3 + 8x_6 - 1772 = 0,\\
f_2(\mathbf{x}) &= 3x_1 + 18x_3 - 5x_5 + 17x_6 - 153 = 0,\\
f_3(\mathbf{x}) &= 6x_1 + x_3 - 99x_2 + (15x_6)^2 - 1772 = 0,\\
f_4(\mathbf{x}) &= -x_1 + 5x_2 + 8x_3 - 6x_4 + 15x_5 + 10x_6 - 277 = 0,\\
f_5(\mathbf{x}) &= (x_1 + x_2)^2 - 7x_3 + 5x_4 + 12x_5 - 8x_6 - 150 = 0,\\
f_6(\mathbf{x}) &= x_2 + 5x_3 - 3x_5 - x_6 - 4 = 0,
\end{aligned}$$

where $\mathbf{x} = (x_1, x_2, \ldots, x_6)^T \in Z^6$ and $D = \{(x_1, x_2, \ldots, x_6) : 1 \le x_i \le 20, i = 1, 2, \ldots, 6\}$. Paper [32] ran its algorithm for 20 times and the running time was ranging from 5.138 to 70.633 s. The SOAC program needs one execution to find the same solution in 0.4602 s, using the parameters values in Table 6. The solutions are shown in Table 7.

Problem 12. From [32], the system of nonlinear equations is following.

$$\begin{aligned}
f_1(\mathbf{x}) &= (x_1)^2 - 2(x_2 + x_4)^3 + x_5 - 3x_6 - x_7 + 4x_9\\
&\quad + 15x_{10} + 24 = 0,\\
f_2(\mathbf{x}) &= 2x_1 + (x_2 + 3x_4)^3 + (5x_7)^2 - 6x_8 + x_9 - 9x_{10} - 31 = 0,\\
f_3(\mathbf{x}) &= 3x_1 - (2x_2)^2 + 10x_3 - 9x_4 + 3x_5 + x_6 - 2x_7 - 8x_8
\end{aligned}$$

Table 5
Solutions for Problems 5–9.

Problems	Given domain and obtained solutions
5(a)	$D = \{(x, n) : 1 \le x \le 500, 3 \le n \le 50\}$, (3, 2, 4), (181, 32, 3), (181, 8, 5), (181, 2, 15), (1, 2, 3), (11, 2, 7), (5, 2, 5).
5(b)	$D = \{(x, y, b) : 1 \le x \le 15000, 1 \le y \le 600, 1 \le b \le 100\}$, (2, 5, 2), (9324, 443, 3), (58, 15, 1), (4, 3, 1).
6(a)	$D = \{(x, y, a, b) : 1 \le x \le 20, 1 \le y \le 20, 1 \le a \le 20, 1 \le b \le 20\}$, (5, 9, 6, 1), (4, 3, 0, 1), (5, 3, 1, 0), (2, 5, 0, 2), (11, 5, 2, 0), (9, 5, 2, 1), (2, 2, 2, 0), (16, 8, 8, 0).
6(b)	$D = \{(x, y, a, b) : 1 \le x \le 20, 1 \le y \le 20, 1 \le a \le 20, 1 \le b \le 20\}$, (0, 1, 0, 0), (0, 2, 4, 0), (0, 4, 8, 0), (0, 11, 0, 4), (7, 3, 5, 0).
7	$D = \{(x, y, k) : 0 \le x \le 50, 0 \le y \le 50, 0 \le k \le 50\}$, (32, 16, 10), (0, 1, 0), (0, 2, 3), (0, 16, 12), (0, 8, 9), (0, 4, 6), (0, 32, 15), (4, 4, 4), (21, 11, 3).
8	$D = \{(x_1, x_2, x_3, x_4) : -1 \le x_i \le 10, i = 1, 2, 3, 4\}$, (0, 0, 0, 0), (2, 2, 0, 2), (5, 1, 6, 4), (1, 5, 6, 4), (1, 1, 1, 1), (1, 1, 2, 0), (3, 1, 4, 2), (1, 3, 4, 2), (3, 3, 5, 1).
9(a)	$D = \{(x, y, z) : 400 \le x \le 500, 1 \le y \le 100, 1 \le z \le 100\}$, (485, 99, 70).
9(b)	$D = \{(x, y, z) : 1 \le x \le 75, 1 \le y \le 75, 1 \le z \le 75\}$, (49, 10, 3).

Table 6
Parameters for Problems 10–12.

Problems	10	11	12
Clustering and selection			
$m_{cluster}$	600	200	30 000
$k_{cluster}$	20	20	20
γ	0.01	0.001	0.01
ϵ	10^{-5}	10^{-7}	10^{-5}
δ	0.01	0.1	0.01
Optimization			
m	50	50	50
k_{max}	30	40	50
r	0.95	0.95	0.95
θ	$\pi/4$	$\pi/4$	$\pi/4$

Table 7
Solutions for Problems 10–12.

Problems	Given domain and obtained solutions
10	$D = \{(x_1, x_2, \ldots, x_7) : -10 \le x_i \le 10, i = 1, 2, \ldots, 7\}$, (1, 3, 2, 2, 0, −3, −5).
11	$D = \{(x_1, x_2, \ldots, x_6) : 1 \le x_i \le 20, i = 1, 2, \ldots, 6\}$, (6, 3, 8, 1, 12, 3).
12	$D = \{(x_1, x_2, \ldots, x_{10}) : 0 \le x_i \le 10, i = 1, 2, \ldots, 10\}$, (3, 4, 5, 0, 1, 1, 1, 2, 2, 6).

$$
\begin{aligned}
&+ 12x_9 - 5x_{10} + 25 = 0,\\
f_4(\mathbf{x}) &= 5x_1 + 2x_2 - 8x_4 - 3x_5 + 4x_6 + x_7 - x_9 - 23 = 0,\\
f_5(\mathbf{x}) &= x_1 - x_3 + 2x_5 - x_7 - x_9 + 3 = 0,\\
f_6(\mathbf{x}) &= x_2 + (2x_4)^2 - 6x_6 - x_8 + 2x_{10} - 8 = 0,\\
f_7(\mathbf{x}) &= 3x_1 + 2x_2 - 5x_3 - (x_4)^4 - 2x_5 + x_6 + 4x_7 - 10x_8\\
&\quad + 8x_9 + 9 = 0,\\
f_8(\mathbf{x}) &= x_1 - 3x_2 + 4x_4 + x_6 - 6x_7 + x_8 - 2x_9 + 16 = 0,\\
f_9(\mathbf{x}) &= (2x_1 + x_2)^2 + 3x_3 - 10x_5 - (x_6 + 3x_7)^3 - x_8\\
&\quad - 6x_9 - 27 = 0,
\end{aligned}
$$

where $\mathbf{x} = (x_1, x_2, \ldots, x_{10})^T \in Z^{10}$ and $D = \{(x_1, x_2, \ldots, x_{10}) : 0 \le x_i \le 10, i = 1, 2, \ldots, 10\}$. Having used parameters values in Table 6 and running time 14.6290 s, the result gives a solution that the same as in [32] and it is shown in Table 7. The running time in [32] was not written clearly. It is said that one of the computations ran for about 36 h. This paper stated that if the width of the search domain increased then the running time would increase exponentially.

4. Discussion on the results

The SOAC method yields more solutions for two problems in Polynomial Diophantine equations, where the reference paper only wrote only one solution for each problem. We expect there are more obtained solutions that are deliberately not written in that paper. Having compared them to the solutions of reference papers for problems in Exponential Diophantine equations, there are additional solutions found using our methods if the conditional *gcd* is not installed. In the problems of the systems of linear and nonlinear Diophantine equations, the results are same with the solution found in the reference papers. We have 0.46 to 14.63 s of the running time, where the reference papers wrote 5.138 to hundreds seconds, and even up to 36 h.

For a comparison on the form of algorithm, a standard PSO algorithm consists of 7 parameters needed for running a computation. Other metaheuristic methods may need smaller or larger number of parameters. The SOAC method needs a larger number of parameters, which are 5 parameters in Clustering and Selection phases and 4 parameters in Optimization phase. Furthermore, we need set the lower and upper bounds of each candidate of solutions, which are called variables. For example, Problem 12 has 9 values of variables to be found, those are x_1 up to x_9.

There is a wild guess that the running time will increase when number of variables increases. In Fig. 1, the running time of all problems are plotted with respect to their numbers of variables. We also plot the numbers of intended clusters ($m_{cluster}$) with respect to the running time in Fig. 2.

Fig. 1 shows unclear relationship between the running time and the number of variables. There are cases with only three or four variables that should be found, but they need more than 90 to 229 s. On the other hand, there are cases with 9 and 10 variables to be found, but they need less than 20 s. If we say more time needed means more difficult the problem, then it seems that the difficulty of the problem is not determined by the number of variables being solved. Any Diophantine problem has its owned type of difficulty. For example, the solution (9324, 443, 3) of Case 5.b and the additional solutions (233, 89, 1) and (433, 29, 5) of Case 4.b are the hardest to be found.

Fig. 2 also shows unclear relationship between the running time and the number of intended clusters. We have Problem 12

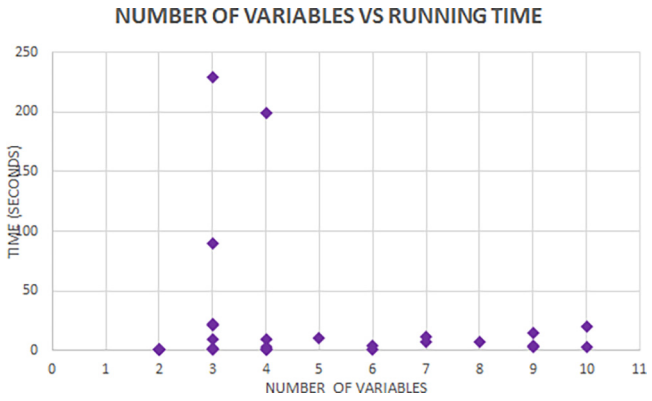

Fig. 1. Plot of the running time with respect to the number of variables.

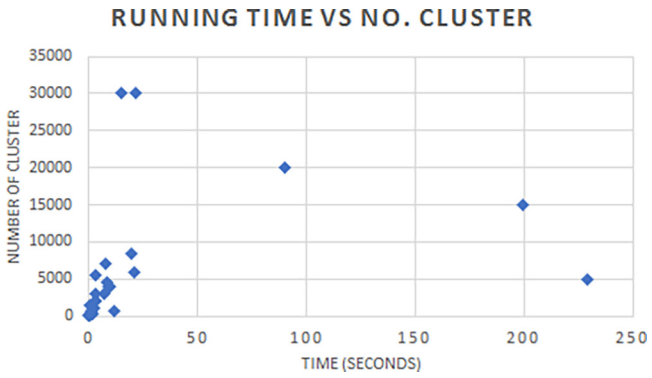

Fig. 2. Plot of the number of clusters with respect to the running time.

where the number of intended clusters is 30,000 with running time 14.6290 s, but Problem 4 needs 5000 clusters and 229.006 s to finish. The determination of the number of clusters is still not automatic, where the user try to pick a larger number if the desired solution has not been found.

In Problems 2, 4, and 6, we can run the SOAC method many times with different lower or upper bounds of the solution's domain, to find different solutions and then collect only the distinct values. However, this contradict with our goal in the first place, which is finding all or almost all solutions in only one execution. Some attempts indeed are required for tuning the appropriate values of the parameters. This problem can be avoid in the first place by defining a long interval of the solution's domain, a large value of $m_{cluster}$, the cluster numbers, and a quite small θ, the rotation angle of spiral algorithm. However, the running time will be very long.

5. Conclusion

In this paper, the results of our method mostly outperform the results from the existing papers. The main benefit of our method is non-random results for every computation, so we get exactly the same solutions if we rerun the method using the same values of parameters. Certainly, the running time will depend on the machine we use. In working on this paper, we only use our home computers or laptops.

Other metaheuristic methods are commonly based on randomness process so they need to rerun many times to get the desired number of distinct solutions. Our SOAC method tries to find all available solutions only in a single run, so we do not need to rerun the simulation to find other possible solutions. This property is due to the existence of the clustering procedure, so the search for solutions performs in parallel inside each cluster.

The number of variables being solved and the number of clusters being used have shown no relation to the time consumption to running the program. The running time depends on the difficulty of the Diophantine equations being solved.

The significant issue to be dealt with using this method is the process of tuning the parameters' values. For solving some difficult problems, we need to change the parameter's value by try and error. When the chosen values are appropriate for the observed case, we will get a more satisfactory result. The more experience you have, the more fluent you set those values. However, in many cases, the tuning of parameters' values is much less time-consuming than experimenting with the type of exhaustion method to find all possible solutions.

We also understand that our algorithm is running in a sequencing order. We can modify the algorithm so it can run in parallel procedures, using a network of different computers, so the performance will be better. So there is a potential use of our algorithm in a parallel processing in a network that give all or almost all solution in a single run.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article

Acknowledgment

We are grateful to 2019 P3MI-ITB Research Grant that support the publication of this work. We thank the unknown reviewers for the valuable input.

References

[1] J. Brüdern, R. Dietmann, Random Diophantine equations, I, Adv. Math. 256 (2014) (2014) 18–45.
[2] L. Sun, Class numbers of quadratic Diophantine equations, J. Number Theory 166 (2016) 181–192.
[3] M. Kazalicki, B. Naskręcki, Diophantine triples and K3 surfaces, J. Number Theory 236 (2022) 41–70.
[4] R. Quinlan, M. Shau, F. Szechtman, Linear Diophantine equations in several variables, Linear Algebra Appl. 640 (2022) 67–90.
[5] C.H. Lin, C.C. Chang, R.C.T. Lee, A new public-key cipher system based upon the Diophantine Equations, IEEE Trans. Comput. 44 (1) (1995) 13–19.
[6] C.S. Laih, M.J. Gau, Cryptanalysis of a Diophantine equation oriented public key cryptosystem, IEEE Trans. Comput. 46 (4) (1997) 511–512.
[7] K.V. Velupillai, Y.F. Kao, Computable and computational complexity theoretic bases for Herbert Simon's cognitive behavioral economics, Cogn. Syst. Res. 29/30 (2014) 40–52.
[8] J.M. Champarnaud, J.P. Dubernard, F. Guingne, H. Jeanne, Geometrical regular languages and linear Diophantine equations: The strongly connected case, Theoret. Comput. Sci. 449 (2012) 54–63.
[9] G. Bocewicz, W. Muszyński, Z. Banaszak, Cyclic scheduling: Diophantine problems perspective, IFAC Proc. Vol. 43 (4) (2010) 338–343.
[10] F. Adiceam, V. Beresnevich, J. Levesley, S. Velani, E. Zorin, Diophantine approximation and applications in interference alignment, Adv. Math. 302 (2016) 231–279.
[11] X. Costa-Pérez, Z. Wu, M. Mezzavilla, J.R.B. de Marca, J. Aráuz, E-diophantine estimating peak allocated capacity in wireless networks, Comput. Commun. 60 (2015) 1–11.
[12] S. Abraham, S. Sanyal, M. Sanglikar, Finding numerical solutions of diophantine equations using Ant Colony Optimization, Appl. Math. Comput. 219 (2013) 11376–11387.
[13] I. Amaya, L. Gomez, R. Correa, Discrete Particle Swarm Optimization in the numerical solution of a system of linear Diophantine Equations, DYNA 81 (185) (2014) 139–144.

[14] K. Tamura, K. Yasuda, Primary study of spiral dyamics inspired optimization, IEEJ Trans. Electr. Electron. Eng. 6 (2010) 99–100.
[15] K. Tamura, K. Yasuda, Spiral dynamics inspired optimization, J. Adv. Comput. Intell. Intell. Inform. 15 (2011) 1116–1122.
[16] L. Benasla, A. Belmadani, M. Rahli, Spiral optimization algorithm for solving combined economic and emission dispatch, Int. J. Electr. Power Energy Syst. 62 (2014) 163–174.
[17] C.W. Tsai, B.C. Huang, M.C. Chiang, A novel spiral optimization for clustering, in: J. Park, H. Adeli, N. Park, I. Woungang (Eds.), Mobile, Ubiquitous, and Intelligent Computing, in: Lecture Notes in Electrical Engineering, vol. 274, Springer, Berlin, Heidelberg, 2014.
[18] M.F. Ansori, K.A. Sidarto, N. Sumarti, Model of deposit and loan of a bank using spiral optimization algorithm, J. Indones. Math. Soc. 25 (3) (2019) 292–301.
[19] A. Kania, K.A. Sidarto, Solving mixed integer nonlinear programming problems using spiral dynamics optimization algorithm, in: Application of Mathematics in Industry and Life, in: AIP Conference Proceeding, vol. 1716, 2016, 020004-1-0200004-9.
[20] K.A. Sidarto, A. Kania, Finding all solutions of systems of nonlinear equations using spiral dynamics optimization with clustering, J. Adv. Comput. Intell. Intell. Inform. 19 (5) (2015) 697–707.
[21] K.A. Sidarto, A. Kania, N. Sumarti, Finding multiple solutions of Multimodal Optimization using Spiral Optimization Algorithm with Clustering, MENDEL — Soft Comput. J. 23 (1) (2017) 95–102.
[22] K.A. Sidarto, A. Kania, Computing complex roots of systems of nonlinear equations using spiral optimization algorithm with clustering, in: R. Alfred, H. Iida, Ag.A. Ibrahim, Y. Lim (Eds.), Computational Science and Technology, in: Lecture Notes in Electrical Engineering, vol. 488, Springer, Singapore, 2018.
[23] V. Aggarwal, Solving transcendental equations using Genetic Algorithms, 2000, http://web.mit.edu/varun_ag/www/ste_gas.pdf. Retrieved in Nov 2019.
[24] K. Tamura, K. Yasuda, A parameter setting method for spiral optimization from stability analysis of dynamics equilibrium point, SICE J. Control Meas. Syst. Integr. 7 (3) (2014) 173–182.
[25] K. Tamura, K. Yasuda, The spiral optimization algorithm: Convergence conditions and settings, IEEE Trans. Syst. Man Cybern.: Syst. 50 (1) (2020).
[26] K.H. Rosen, Elementary Number Theory and Its Application, Addison-Wesley Publishing Company, 1984.
[27] S. Abraham, S. Sanyal, M. Sanglikar, A connectionist network approach to find numerical solutions of Diophantine equations, Int. J. Eng. Sci. Manag. III (1) (2013) 73–81.
[28] S. Gao, K.H. Chen, Computation in Markoff-Hurwitz equations, in: 2016 International Conference on Computational Science and Computational Intelligence, CSCI, 2016, pp. 1292–1297.
[29] G. Soydan, On the Diophantine equation $x^2 + 7^a \dot{1}1^b = y^n$, 2012, https://arxiv.org/pdf/1201.0778.pdf. Retrieved in July 2019.
[30] I.N. Cangul, M. Demirci, F. Luca, A. Pinter, G. Soydan, On the Diophantine equation $x^2 + 2^a.11^b = y^n$, Fibonacci Quart. 48 (2010) 39–46.
[31] F. Luca, G. Soydan, On the Diophantine $2m + nx^2 = y^n$, J. Number Theory 132 (2012) 2604–2609.
[32] O. Perez, I. Amaya, R. Correa, Numerical solution of certain exponential and nonlinear Diophantine systems of equation by using a discrete particle swarm optimization algorithm, Appl. Math. Comput. 225 (2013) 737–746.
[33] X. Ai, J. Chen, S. Zhang, H. Hu, Complete solution of the simultaneous Pell equation $x^2 - 24y^2 = 1$ and $y^2 - pz^2 = 1$, J. Number Theory 147 (2015) 103–108.